

ĐỒ ÁN HỌC PHẦN
LẬP TRÌNH MẠNG MÁY TÍNH
XÂY DỰNG BLOG CÁ NHÂN CHIA SẺ KIẾN THỨC
LẬP TRÌNH MẠNG
(Ứng dụng Hugo Static Site Generator và GitHub Pages)

Ngành: **CÔNG NGHỆ THÔNG TIN**

Chuyên ngành: **CÔNG NGHỆ PHẦN MỀM**

Giảng viên hướng dẫn: **ThS. Nguyễn Quang Trung**

Sinh viên thực hiện:

MSSV:

Lớp:

1. Ngô Phạm Ngọc Tú

2280603518

22DTHE5

TP. Hồ Chí Minh, 2025

MỤC LỤC

LỜI NÓI ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI.....	2
1.1. Lý do chọn đề tài.....	2
1.2. Yêu cầu đề tài	2
1.3. Nội dung đề tài.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	4
2.1. Tổng quan về Static Site Generator (SSG) và Hugo	4
2.2. Markdown và Front Matter	4
2.3. Các kiến thức Lập trình mạng áp dụng	4
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	6
3.1. Cấu trúc dự án Hugo	6
3.2. Lựa chọn Giao diện (Theme PaperMod).....	6
3.3. Thiết kế cấu trúc nội dung	7
CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI BLOG CÁ NHÂN	8
4.1. Cài đặt môi trường và Khởi tạo dự án	8
4.2. Cấu hình Theme và Thông tin cá nhân	8
4.3. Xây dựng nội dung chuyên môn (Blog & Profile).....	9
4.4. Kết quả hiển thị thực tế	10
4.4.1. Trang chủ (Home Page):	10
4.4.2. Trang Profile cá nhân (CV):	11
4.4.3. Chi tiết bài viết (Post Detail):	13
4.4.4. Chế độ tối (Dark Mode):	15
CHƯƠNG 5. KẾT LUẬN.....	16
5.1. Kết quả đạt được	16
5.2. Định hướng phát triển	16
TÀI LIỆU THAM KHẢO	17
PHỤ LỤC: NỘI DUNG CHI TIẾT CÁC BÀI VIẾT	18

DANH MỤC HÌNH

Hình 3.1: Ảnh Cấu trúc thư mục dự án.....	6
Hình 3.2: Sơ đồ Site Map hoặc Menu điều hướng	7
Hình 4.1: Ảnh: Cấu hình file hugo.toml	9
Hình 4.2: Ảnh: Danh sách bài viết trong thư mục code.....	10
Hình 4.3: Ảnh Trang chủ Website	11
Hình 4.4: Ảnh profile chi tiết cv	12
Hình 4.5: Ảnh chứng đã hoàn thành	13
Hình 4.6: Ảnh Blog cá nhân.....	14
Hình 4.7: ảnh Nội dung bài post	14
Hình 4.8: Ảnh giao diện website sáng, tối	15

LỜI NÓI ĐẦU

Trong kỷ nguyên số hiện nay, việc xây dựng thương hiệu cá nhân và chia sẻ kiến thức là kỹ năng mềm quan trọng đối với một kỹ sư Công nghệ thông tin tương lai. Đặc biệt trong lĩnh vực Lập trình mạng, công nghệ thay đổi nhanh chóng đòi hỏi người học phải có khả năng tự học, tổng hợp và lưu trữ kiến thức một cách khoa học.

Đồ án học phần "**Lập trình mạng máy tính**" là cơ hội quý báu để em không chỉ ôn tập lại các kiến thức cốt lõi như Socket, HTTP, REST API mà còn thực hành kỹ năng xây dựng website, quản lý mã nguồn và triển khai ứng dụng thực tế.

Em xin chân thành cảm ơn **ThS. Nguyễn Quang Trung** đã tận tình giảng dạy, truyền đạt kiến thức và hướng dẫn em trong suốt quá trình học tập môn Lập trình mạng. Những kiến thức Thầy truyền đạt là nền tảng vững chắc để em hoàn thành đồ án này.

Đồ án "**Xây dựng Blog cá nhân chia sẻ kiến thức Lập trình mạng**" là kết quả của quá trình nỗ lực tìm hiểu và thực hành nghiêm túc. Tuy nhiên, do giới hạn về thời gian và kiến thức, báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự đóng góp ý kiến của Thầy để hoàn thiện hơn.

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Là sinh viên năm 4 chuyên ngành Công nghệ phần mềm, em nhận thấy việc lưu trữ kiến thức dưới dạng các ghi chú rời rạc trên máy tính cá nhân không mang lại hiệu quả cao khi cần tra cứu hoặc chia sẻ. Bên cạnh đó, các nền tảng mạng xã hội không tối ưu cho việc trình bày các bài viết kỹ thuật chứa nhiều đoạn mã (code block).

Việc xây dựng một Blog cá nhân (Personal Blog) mang lại nhiều lợi ích:

1. **Hệ thống hóa kiến thức:** Giúp ôn lại sâu sắc các khái niệm Lập trình mạng đã học.
2. **Xây dựng Portfolio:** Tạo ấn tượng chuyên nghiệp với nhà tuyển dụng thông qua CV trực tuyến.
3. **Làm chủ công nghệ:** Tiếp cận với các công cụ hiện đại như Git, Hugo (Static Site Generator) và Markdown.

1.2. Yêu cầu đề tài

Đồ án cần đáp ứng các yêu cầu sau:

- **Về nền tảng:** Sử dụng Framework tạo trang tĩnh (SSG), cụ thể là **Hugo**.
- **Về mã nguồn:** Quản lý source code bằng **Git** và lưu trữ trên **GitHub**.
- **Về nội dung:**
 - Trang chủ (Home) giới thiệu tổng quan.
 - Trang Profile/CV giới thiệu bản thân, kỹ năng và học vấn.
 - Tối thiểu 9 bài viết chất lượng cao (Technical Posts) về chủ đề Lập trình mạng (Java, JS, Protocols).
- **Về giao diện:** Tối giản, tập trung vào nội dung đọc, hỗ trợ thiết bị di động (Responsive) và chế độ tối (Dark Mode).

1.3. Nội dung đề tài

Phạm vi đề án tập trung vào việc:

1. Cài đặt và cấu hình Hugo Framework.
2. Tích hợp và tùy biến giao diện **PaperMod**.
3. Biên soạn nội dung kiến thức Lập trình mạng (Socket Java, REST API, Async/Await...).
4. Triển khai website lên môi trường Internet (GitHub Pages).

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Tổng quan về Static Site Generator (SSG) và Hugo

Static Site Generator (SSG) là công cụ giúp tạo ra các trang web HTML tĩnh từ dữ liệu đầu vào (thường là file Markdown) và các template. Không giống như WordPress hay các CMS động cần cơ sở dữ liệu (Database), SSG tạo ra file HTML sẵn sàng để phục vụ ngay lập tức.

Hugo là một trong những SSG phổ biến nhất thế giới hiện nay, được viết bằng ngôn ngữ **Go** (Golang).

- **Ưu điểm:** Tốc độ build cực nhanh (mili-giây), bảo mật cao, chi phí vận hành thấp (có thể host miễn phí trên GitHub Pages).

2.2. Markdown và Front Matter

Toàn bộ nội dung trên Blog được viết bằng định dạng **Markdown (.md)**. Đây là ngôn ngữ đánh dấu nhẹ, giúp lập trình viên tập trung vào nội dung thay vì định dạng phức tạp.

Mỗi bài viết bắt đầu bằng khối **Front Matter** (định dạng TOML hoặc YAML) để khai báo siêu dữ liệu:

```
+++  
title = "Tên bài viết"  
date = "2025-12-20"  
tags = ["Java", "Socket"]  
categories = ["Lập trình mạng"]  
+++
```

2.3. Các kiến thức Lập trình mạng áp dụng

Nội dung Blog được xây dựng dựa trên các kiến thức chuyên ngành đã nghiên cứu:

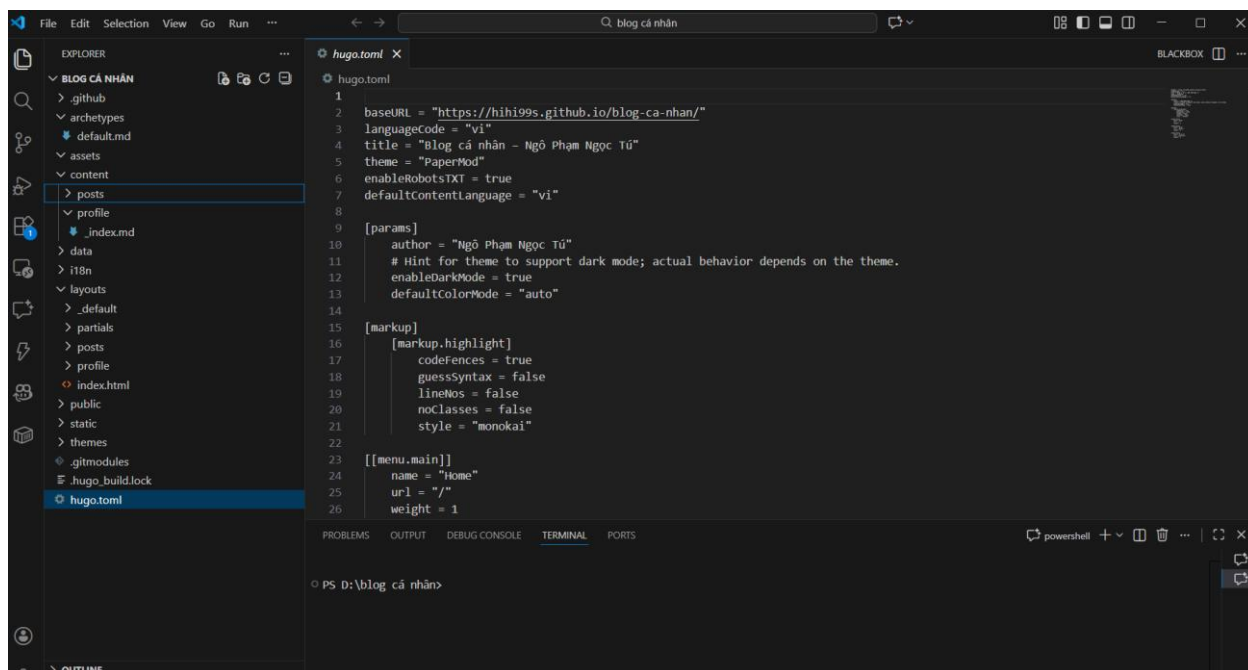
- **Giao thức TCP/UDP:** Sự khác biệt về độ tin cậy, hướng kết nối và ứng dụng thực tế (Chat, Streaming).

- **Socket Programming:** Kỹ thuật lập trình giao tiếp giữa Client và Server trong Java sử dụng `java.net.Socket`.
- **HTTP & REST API:** Các phương thức (GET, POST, PUT, DELETE), mã trạng thái (Status Code) và kiến trúc RESTful.
- **Asynchronous Processing:** Xử lý bất đồng bộ trong JavaScript (Promise, Async/Await) để tối ưu hóa hiệu năng ứng dụng mạng.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Cấu trúc dự án Hugo

Dựa trên mã nguồn thực tế của dự án, cấu trúc thư mục được tổ chức như sau:



Hình 3.1: Ảnh Cấu trúc thư mục dự án

- **hugo.toml**: File cấu hình trung tâm của website.
- **content/**: Chứa toàn bộ nội dung bài viết và trang profile.
 - **posts/**: Chứa 9 bài blog kỹ thuật (socket-java.md, rest-api.md...).
 - **profile/**: Chứa file `_index.md` hiển thị CV cá nhân.
- **themes/PaperMod/**: Mã nguồn giao diện, được add dưới dạng Git Submodule.
- **static/**: Chứa tài nguyên tĩnh như hình ảnh (images/), CSS tùy chỉnh (css/custom.css).

3.2. Lựa chọn Giao diện (Theme PaperMod)

Em lựa chọn **PaperMod** vì đây là theme được đánh giá rất cao trong cộng đồng Hugo:

1. **Hiệu năng**: Điểm Google PageSpeed đạt 100/100.

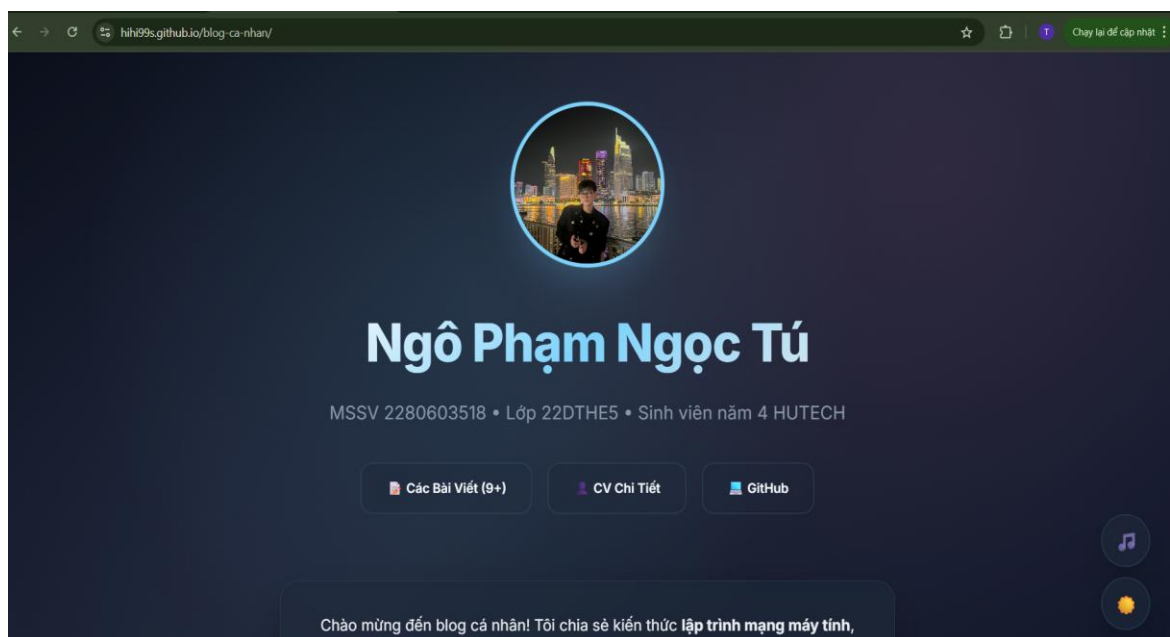
2. **Tính năng:** Tự động chuyển chế độ Sáng/Tối (Dark Mode), hỗ trợ tìm kiếm (Search), chia sẻ mạng xã hội.

3. **Thẩm mỹ:** Thiết kế tối giản (Minimalist), làm nổi bật các đoạn code (Syntax Highlighting) - rất phù hợp cho blog lập trình.

3.3. Thiết kế cấu trúc nội dung

Website được chia thành các phân vùng chính:

- **Menu điều hướng:** Home, Blog, Profile, Search.
- **Profile Page:** Hiển thị ảnh đại diện, thông tin sinh viên, kỹ năng và chứng chỉ.
- **Blog Listing:** Danh sách bài viết được sắp xếp theo thời gian mới nhất.



Hình 3.2: Sơ đồ Site Map hoặc Menu điều hướng

CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI BLOG CÁ NHÂN

4.1. Cài đặt môi trường và Khởi tạo dự án

Sau khi cài đặt Git và Hugo Extended, em tiến hành khởi tạo dự án và cài đặt theme:

```
hugo new site blog-ca-nhan
cd blog-ca-nhan
git init
git submodule add [https://github.com/adityatelange/hugo-
PaperMod](https://github.com/adityatelange/hugo-PaperMod)
themes/PaperMod
```

4.2. Cấu hình Theme và Thông tin cá nhân

File hugo.toml được cấu hình để hiển thị thông tin cá nhân và thiết lập các tham số cho theme PaperMod:

- **BaseURL:** <https://hihi99s.github.io/blog-ca-nhan/>
- **Title:** Blog cá nhân – Ngô Phạm Ngọc Tú
- **Menu:** Cấu hình thanh điều hướng trở về /posts/ và /profile/.

```

1  hugo.toml
2  baseURL = "https://hihi99s.github.io/blog-ca-nhan/"
3  languageCode = "vi"
4  title = "Blog cá nhân - Ngô Phạm Ngọc Tú"
5  theme = "PaperMod"
6  enableRobotsTXT = true
7  defaultContentLanguage = "vi"
8
9  [params]
10     author = "Ngô Phạm Ngọc Tú"
11     # Hint for theme to support dark mode; actual behavior depends on the theme.
12     enableDarkMode = true
13     defaultColorMode = "auto"
14
15  [markup]
16    [markup.highlight]
17      codeFences = true
18      guessSyntax = false
19      lineNos = false
20      noClasses = false
21      style = "monokai"
22
23  [[menu.main]]
24    name = "Home"
25    url = "/"
26    weight = 1
27
28  [[menu.main]]
29    name = "Blog"
30    url = "/posts/"
31    weight = 2
32
33  [[menu.main]]
34    name = "Profile"
35    url = "/profile/"
36    weight = 3

```

Hình 4.1: Ảnh: Cấu hình file hugo.toml

4.3. Xây dựng nội dung chuyên môn (Blog & Profile)

Em đã hoàn thành việc biên soạn 9 bài viết chuyên sâu và 1 trang CV, sử dụng cú pháp Markdown kết hợp HTML để trình bày đẹp mắt.

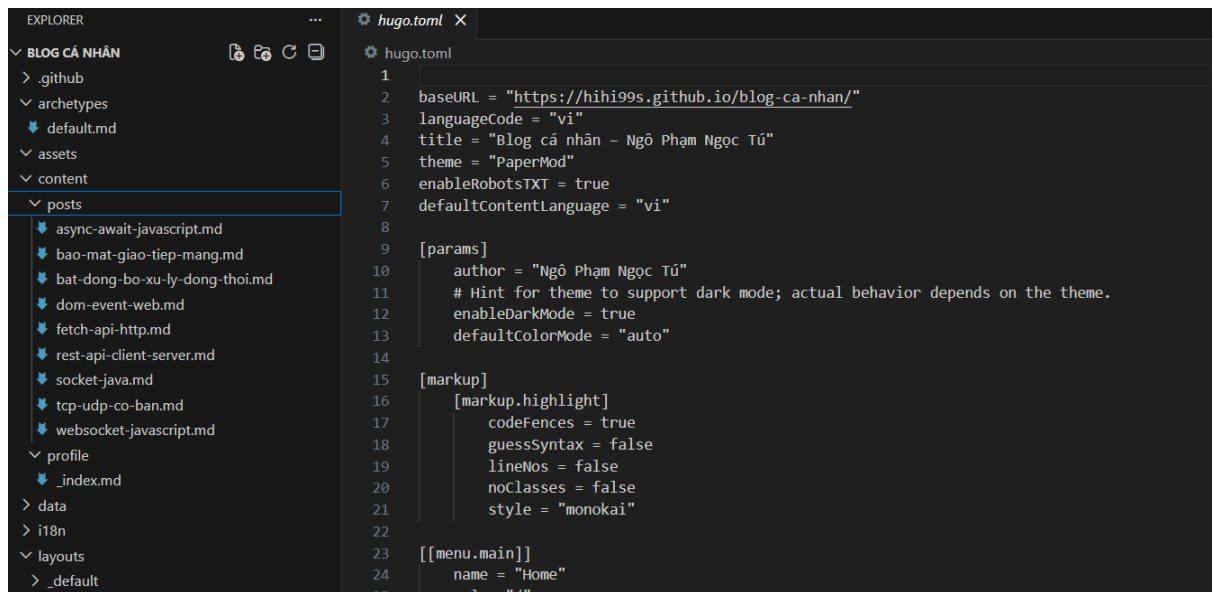
Trang Profile (content/profile/_index.md):

Giới thiệu chi tiết về bản thân, bao gồm các thông tin:

- Họ tên: Ngô Phạm Ngọc Tú
- MSSV: 2280603518
- Kỹ năng: Java, JavaScript, Git.
- Chứng chỉ: Cisco Networking Academy (Networking Basics, JS Essentials).

Các bài viết Blog (content/posts/):

1. *Socket Programming trong Java*: Hướng dẫn tạo Client-Server chat.
2. *Async/Await trong JavaScript*: Xử lý bất đồng bộ hiện đại.
3. *REST API và Kiến trúc Client-Server*: Nguyên lý thiết kế API.
4. *TCP và UDP*: So sánh giao thức vận chuyển.
5. Bảo mật cơ bản trong Giao tiếp Mạng: HTTPS, SSL/TLS.
- ... và các bài viết khác về Fetch API, DOM, WebSocket.



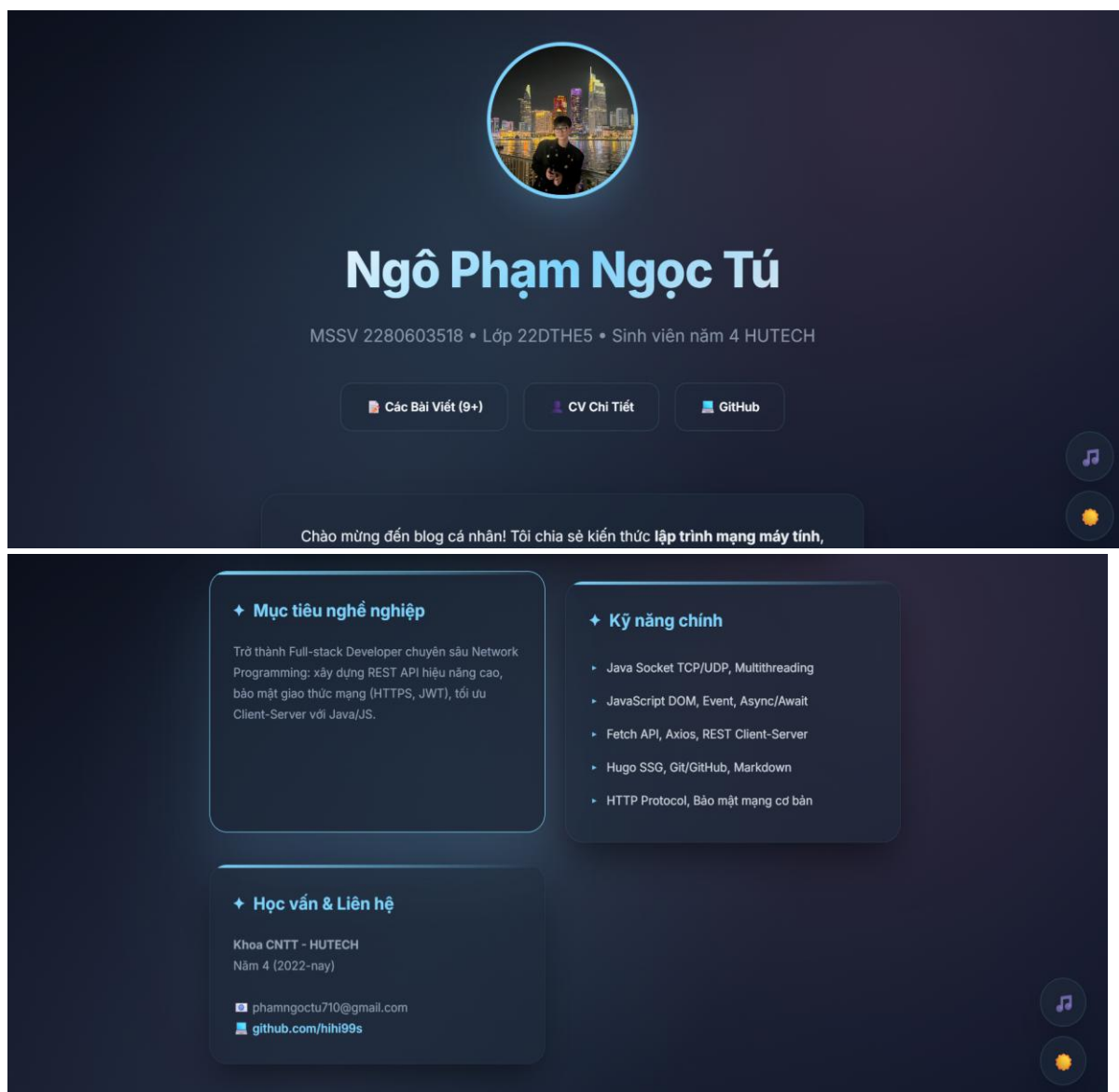
Hình 4.2: Ảnh: Danh sách bài viết trong thư mục code

4.4. Kết quả hiển thị thực tế

Dưới đây là hình ảnh thực tế của Blog sau khi chạy lệnh hugo server và triển khai.

4.4.1. Trang chủ (Home Page):

Giao diện hiển thị danh sách bài viết mới nhất cùng thanh menu điều hướng.



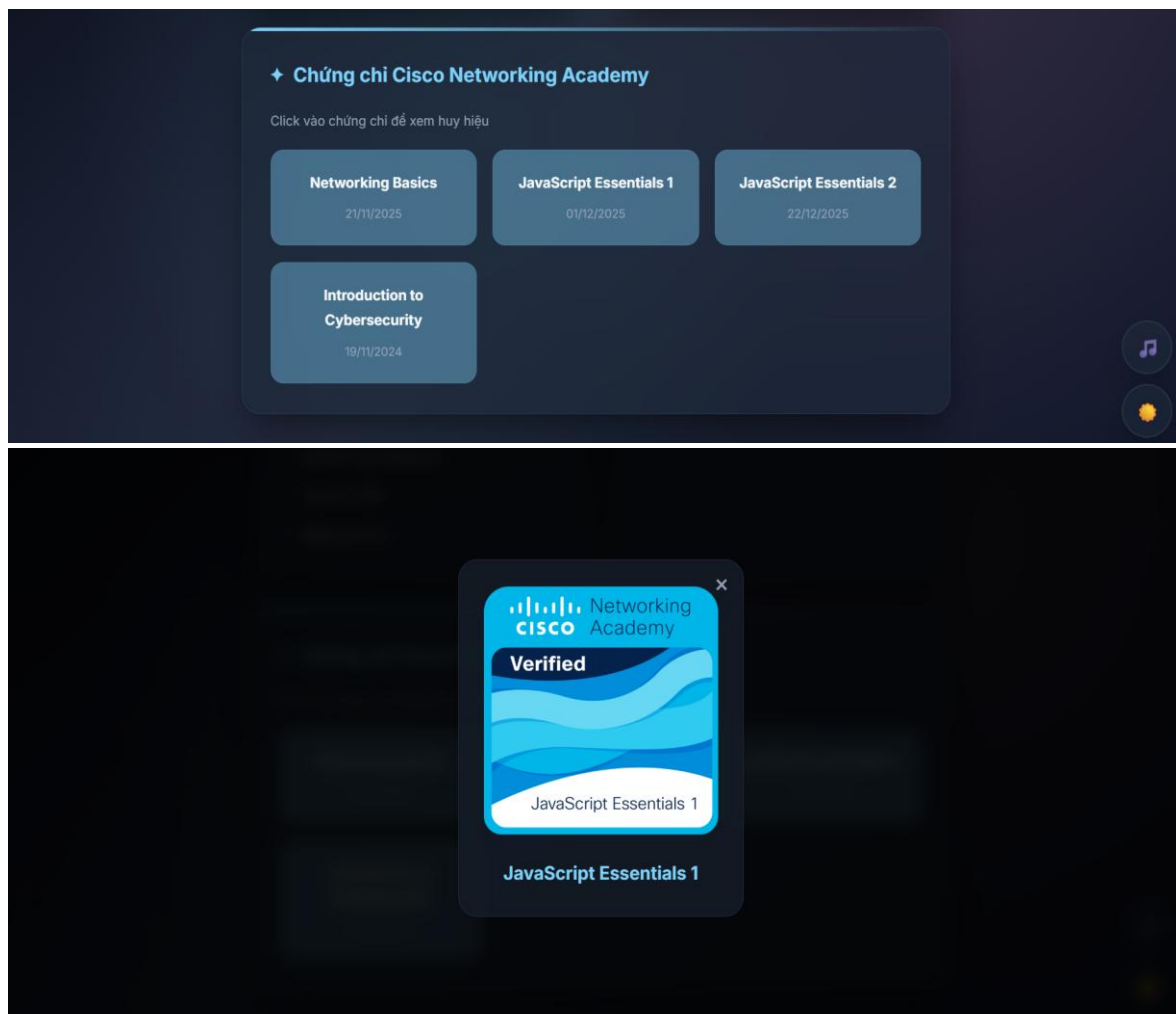
Hình 4.3: Ảnh Trang chủ Website

4.4.2. Trang Profile cá nhân (CV):

Hiển thị thông tin sinh viên, bảng điểm kỹ năng và các chứng chỉ đạt được.



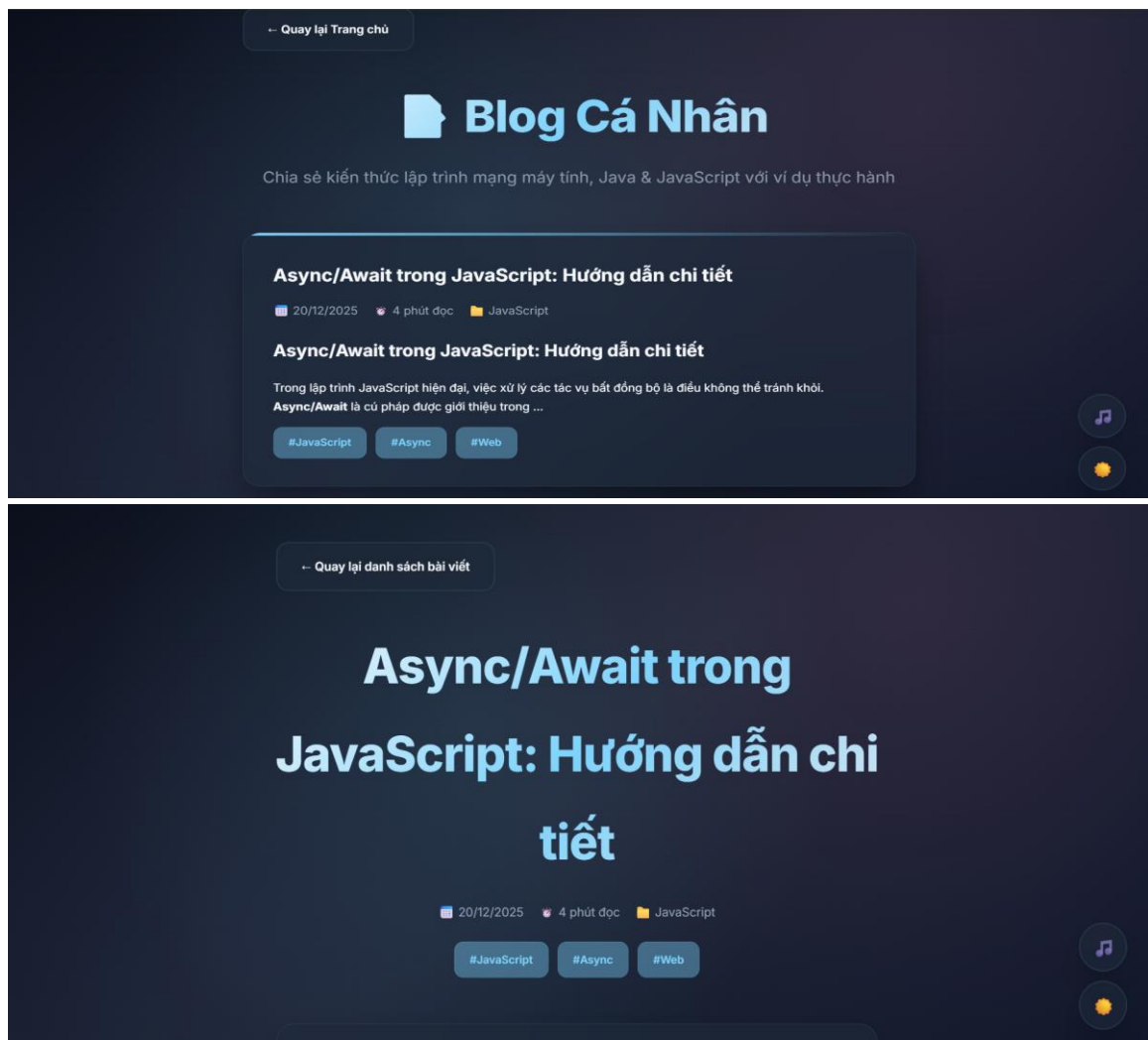
Hình 4.4: Ảnh profile chi tiết cv



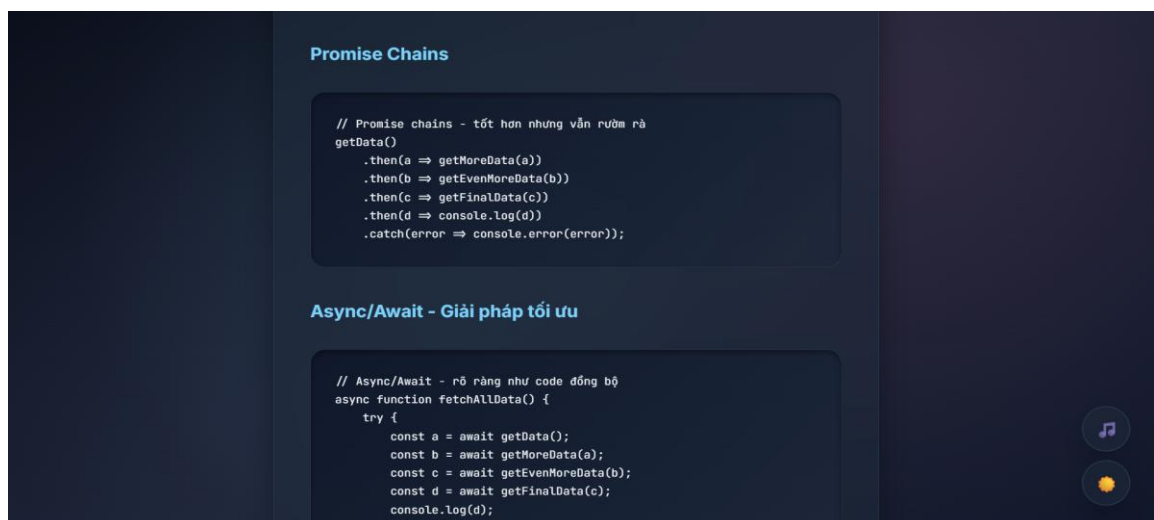
Hình 4.5: Ảnh chứng đã hoàn thành

4.4.3. Chi tiết bài viết (Post Detail):

Giao diện bài viết "Socket Programming trong Java" với khả năng highlight code Java.



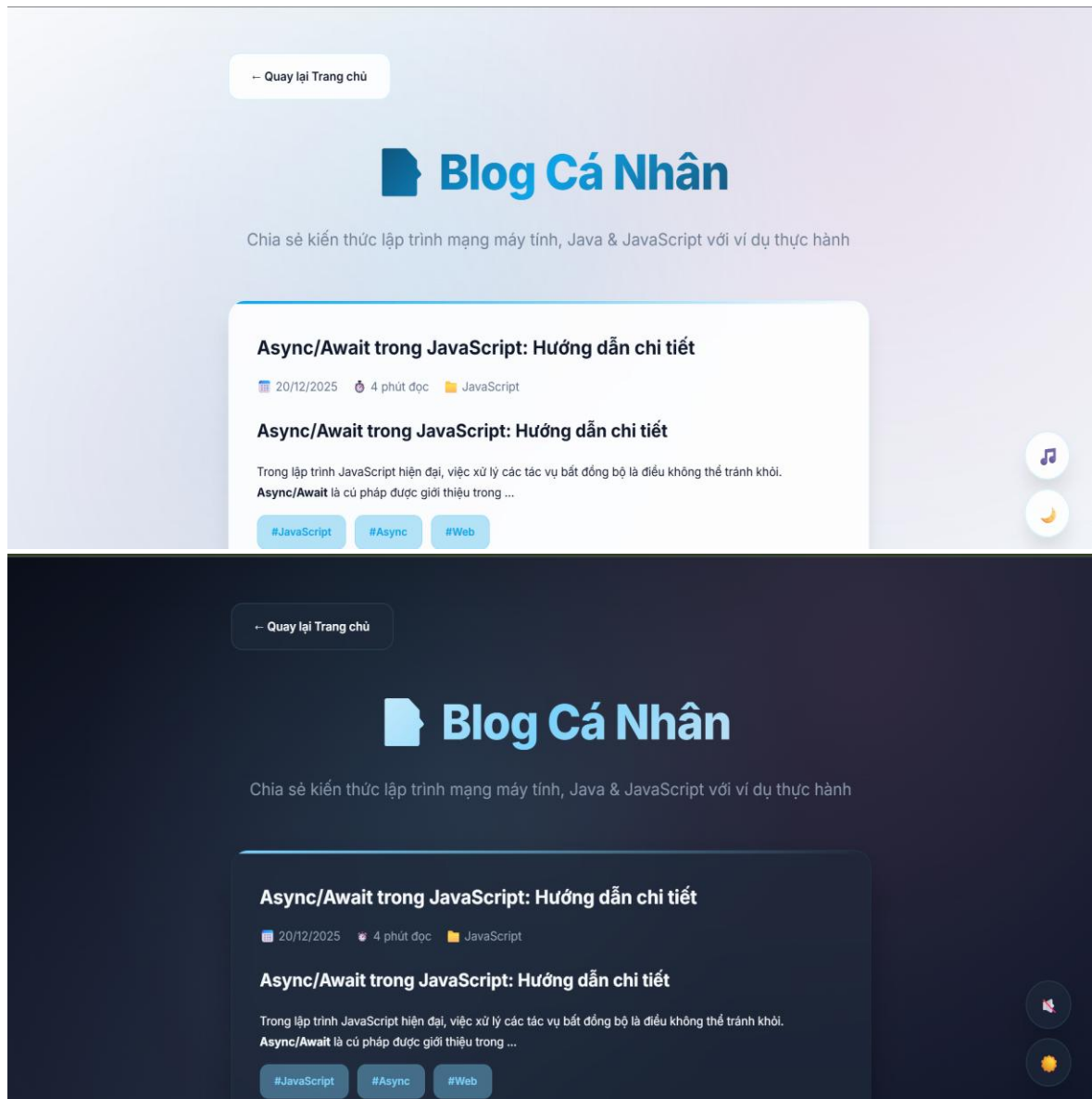
Hình 4.6: Ảnh Blog cá nhân



Hình 4.7: ảnh Nội dung bài post

4.4.4. Chế độ tối (Dark Mode):

Giao diện tự động chuyển sang nền tối khi người dùng cài đặt hệ thống là Dark Mode.



Hình 4.8: Ảnh giao diện website sáng, tối

CHƯƠNG 5. KẾT LUẬN

5.1. Kết quả đạt được

Sau quá trình thực hiện đồ án, em đã hoàn thành tốt các mục tiêu đề ra ban đầu:

1. Xây dựng thành công Blog cá nhân hoàn chỉnh tại địa chỉ: <https://hihi99s.github.io/blog-ca-nhan/>.
2. Nắm vững quy trình phát triển website tĩnh sử dụng **Hugo** và **GitHub Pages**.
3. Hệ thống hóa được khối lượng lớn kiến thức Lập trình mạng thông qua việc viết 9 bài blog chuyên sâu.
4. Có được một CV trực tuyến chuyên nghiệp để phục vụ cho công việc sau này.

5.2. Định hướng phát triển

Trong tương lai, em dự định sẽ nâng cấp Blog với các tính năng:

- **Tích hợp bình luận (Comments):** Sử dụng Giscus hoặc Utterances để người đọc có thể thảo luận.
- **Analytics:** Tích hợp Google Analytics để theo dõi lượt truy cập.
- **Nội dung nâng cao:** Viết thêm về các chủ đề Microservices, Docker và CI/CD trong Lập trình mạng.

TÀI LIỆU THAM KHẢO

1. **Hugo Documentation** - *gohugo.io*

2. **PaperMod Theme Wiki** -

<https://www.google.com/search?q=github.com/adityatelange/hugo-PaperMod>

3. **Văn Thiên Hoàng (2021)**. *Lập trình mạng máy tính*. HUTECH.

4. **Mozilla Developer Network (MDN)** - *developer.mozilla.org*

5. **Oracle Java Documentation** - *docs.oracle.com*

<div style="page-break-after: always;"></div>

PHỤ LỤC: NỘI DUNG CHI TIẾT CÁC BÀI VIẾT

Dưới đây là nội dung chi tiết của 9 bài viết chuyên môn đã được đăng tải trên Blog.

Bài 1: Socket Programming trong Java: Hướng dẫn chi tiết

Socket là endpoint của kết nối mạng hai chiều giữa hai chương trình chạy trên mạng. Trong Java, package `java.net` cung cấp các class để làm việc với socket một cách dễ dàng.

Khái niệm cơ bản

Socket là gì?

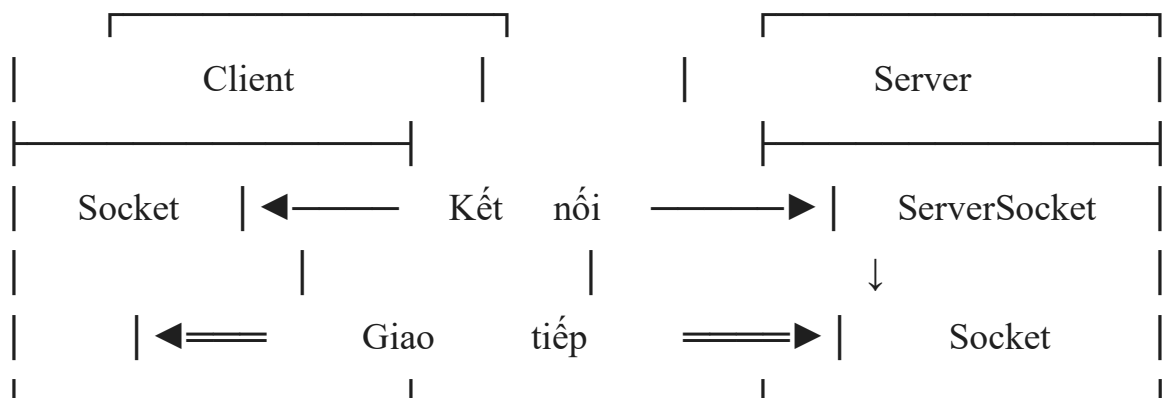
Socket là sự kết hợp của:

- **IP Address:** Xác định máy tính trên mạng
- **Port Number:** Xác định ứng dụng trên máy tính (0-65535)

Socket = IP Address + Port Number

Ví dụ: 192.168.1.100:8080

Mô hình Client-Server



TCP Socket trong Java

Server Side

```
import java.net.*;
```

```

import java.io.*;

public class TCPServer {
    public static void main(String[] args) {
        int port = 8080;

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server đang lắng nghe tại port " + port);

            while (true) {
                // Chờ client kết nối
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client đã kết nối: " +
                    clientSocket.getInetAddress().getHostAddress());

                // Xử lý client trong thread riêng
                new ClientHandler(clientSocket).start();
            }

        } catch (IOException e) {
            System.err.println("Lỗi server: " + e.getMessage());
        }
    }
}

class ClientHandler extends Thread {
    private Socket clientSocket;

```

```

    public ClientHandler(Socket socket) {
        this.clientSocket = socket;
    }

    @Override
    public void run() {
        try (
            BufferedReader in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
            PrintWriter out = new PrintWriter(
                clientSocket.getOutputStream(), true)
        ) {
            String inputLine;
            while ((inputLine = in.readLine()) != null) {
                System.out.println("Nhận được: " + inputLine);

                // Echo back với xử lý
                String response = "Server nhận: " + inputLine.toUpperCase();
                out.println(response);

                if ("bye".equalsIgnoreCase(inputLine)) {
                    break;
                }
            }

            } catch (IOException e) {
                System.err.println("Lỗi xử lý client: " + e.getMessage());
            }
    }

```

```

    } finally {
    try {
    clientSocket.close();
    } catch (IOException e) {
    e.printStackTrace();
    }
    }
    }
}

```

Client Side

```

import java.net.*;
import java.io.*;
import java.util.Scanner;

public class TCPClient {
    public static void main(String[] args) {
        String serverAddress = "localhost";
        int port = 8080;

        try (
            Socket socket = new Socket(serverAddress, port);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            Scanner scanner = new Scanner(System.in)
        ) {

```



```

System.out.println("Đã kết nối tới server!");
System.out.println("Nhập tin nhắn (gõ 'bye' để thoát:");

String userInput;
while (true) {
    System.out.print("Bạn: ");
    userInput = scanner.nextLine();

    // Gửi tới server
    out.println(userInput);

    // Nhận phản hồi
    String response = in.readLine();
    System.out.println("Server: " + response);

    if ("bye".equalsIgnoreCase(userInput)) {
        break;
    }
}

} catch (UnknownHostException e) {
    System.err.println("Không tìm thấy server: " + serverAddress);
} catch (IOException e) {
    System.err.println("Lỗi I/O: " + e.getMessage());
}
}
}
}

```

UDP Socket trong Java

UDP Server

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) {
        int port = 9090;
        byte[] buffer = new byte[1024];

        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Server đang chờ tại port " + port);

            while (true) {
                // Nhận datagram
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);

                String message = new String(request.getData(), 0, request.getLength());
                System.out.println("Nhận từ " + request.getAddress() + ": " + message);

                // Gửi phản hồi
                String response = "Echo: " + message;
                byte[] responseData = response.getBytes();
                DatagramPacket responsePacket = new DatagramPacket(
                    responseData,
                    responseData.length,
                    request.getAddress(),
```

```

request.getPort()
);
socket.send(responsePacket);
}

} catch (Exception e) {
System.err.println("Lỗi: " + e.getMessage());
}
}
}
}

```

Các class quan trọng trong java.net

Class	Mô tả
Socket	TCP client socket
ServerSocket	TCP server socket
DatagramSocket	UDP socket
DatagramPacket	UDP packet
InetAddress	Địa chỉ IP
URL	Uniform Resource Locator
URLConnection	Kết nối tới URL

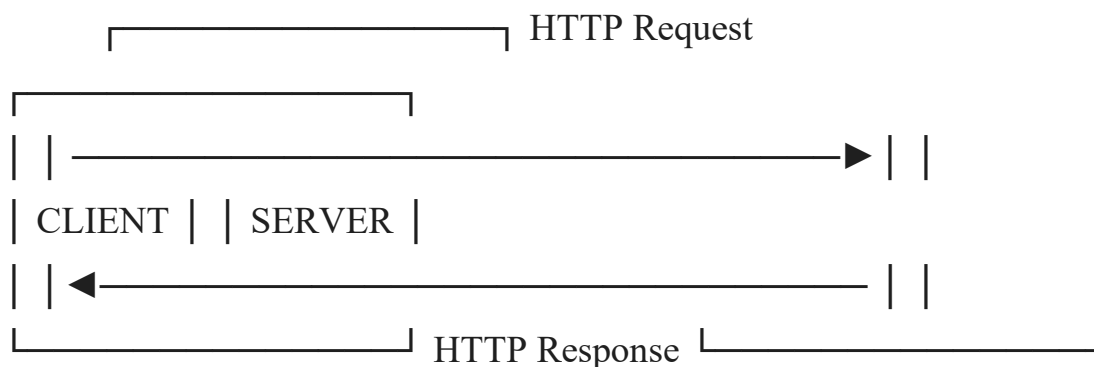
<div style="page-break-after: always;"></div>

Bài 2: REST API và Kiến trúc Client-Server

REST (Representational State Transfer) là kiến trúc phổ biến nhất để xây dựng API cho ứng dụng web. Bài viết này sẽ giải thích REST API và mô hình Client-Server một cách chi tiết.

Kiến trúc Client-Server

Mô hình cơ bản



(Browser, (API,
Mobile App, Database,
Desktop App) Business Logic)

Đặc điểm

- **Client:** Gửi request, hiển thị dữ liệu cho user
- **Server:** Xử lý logic, lưu trữ dữ liệu, trả response
- **Stateless:** Server không lưu trạng thái client giữa các request
- **Separation of Concerns:** Tách biệt UI và logic

REST là gì?

REST là một **kiến trúc** (không phải giao thức) với các nguyên tắc thiết kế API:

6 Ràng buộc của REST

1. **Client-Server:** Tách biệt client và server
2. **Stateless:** Mỗi request chứa đủ thông tin, server không lưu session

3. **Cacheable:** Response có thể được cache
4. **Uniform Interface:** Giao diện thống nhất
5. **Layered System:** Có thể có các layer trung gian (proxy, load balancer)
6. **Code on Demand** (optional): Server có thể gửi code để client thực thi

HTTP Methods trong REST

Method	Mục đích	Ví dụ	Idempotent
GET	Lấy dữ liệu	Lấy danh sách users	Có
POST	Tạo mới	Tạo user mới	Không
PUT	Cập nhật toàn bộ	Thay thế thông tin user	Có
PATCH	Cập nhật một phần	Đổi email user	Có
DELETE	Xóa	Xóa user	Có

Thiết kế URL RESTful

Quy tắc đặt tên

✓ Tốt:

GET /users - Lấy tất cả users

GET /users/123 - Lấy user id 123

POST /users - Tạo user mới

PUT /users/123 - Cập nhật user 123

DELETE /users/123 - Xóa user 123

GET /users/123/posts - Lấy posts của user 123

GET /posts?author=123 - Lấy posts với filter

✗ Không tốt:

GET /getUsers

GET /user/get/123

POST /createNewUser

GET /getAllPostsOfUser/123

HTTP Status Codes

Code	Ý nghĩa
200	OK - Request thành công
201	Created - Tạo mới thành công
400	Bad Request - Request không hợp lệ
401	Unauthorized - Chưa xác thực
404	Not Found - Không tìm thấy
500	Internal Server Error

Ví dụ Client JavaScript với Fetch

```
const API_BASE = '[https://api.example.com](https://api.example.com)';
```

```
// API Service
```

```
const ProductAPI = {
```

```
  // Lấy tất cả sản phẩm
```

```
  async getAll(params = {}) {
```

```
    const query = new URLSearchParams(params).toString();
```

```
    const url = `${API_BASE}/products${query ? '?' + query : ''}`;
```

```

    const response = await fetch(url);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  },

  // Tạo sản phẩm mới
  async create(productData) {
    const response = await fetch(`${API_BASE}/products`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(productData)
    });

    return response.json();
  }
};

```

<div style="page-break-after: always;"></div>

Bài 3: Async/Await trong JavaScript: Hướng dẫn chi tiết

Trong lập trình JavaScript hiện đại, việc xử lý các tác vụ bất đồng bộ là điều không thể tránh khỏi. **Async/Await** là cú pháp được giới thiệu trong ES2017 (ES8) giúp code bất đồng bộ trở nên dễ đọc và dễ bảo trì hơn rất nhiều.

Tại sao cần Async/Await?

Trước khi có Async/Await, chúng ta phải sử dụng **callback** hoặc

Promise chains, dẫn đến code khó đọc.

Callback Hell (Địa ngục callback)

```
getData(function(a) {  
  getMoreData(a, function(b) {  
    getEvenMoreData(b, function(c) {  
      console.log(c);  
    });  
  });  
});
```

Async/Await - Giải pháp tối ưu

// Async/Await - rõ ràng như code đồng bộ

```
async function fetchAllData() {  
  try {  
    const a = await getData();  
    const b = await getMoreData(a);  
    const c = await getEvenMoreData(b);  
    console.log(c);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Cách hoạt động của Async/Await

Từ khóa async

Khi đặt **async** trước một function, function đó sẽ **luôn trả về một**

Promise:


```
    async function sayHello() {  
    return "Xin chào!";  
}
```

// Tương đương với:

```
function sayHello() {  
    return Promise.resolve("Xin chào!");  
}
```

Từ khóa await

await chỉ được sử dụng **bên trong async function** và nó sẽ:

- Tạm dừng thực thi function
- Chờ Promise hoàn thành
- Trả về giá trị resolved của Promise

```
    async function demo() {  
    console.log("Bắt đầu");
```

// Tạm dừng 2 giây

```
    await new Promise(resolve => setTimeout(resolve, 2000));
```

```
    console.log("Sau 2 giây");
```

```
}
```

Xử lý nhiều Promise song song

Tuần tự (Sequential) - Chậm

```
    async function tuanTu() {  
    const user = await fetch('/api/user'); // Chờ xong mới chạy tiếp  
    const posts = await fetch('/api/posts'); // Chờ xong mới chạy tiếp  
}
```

Song song (Parallel) - Nhanh

```
async function songSong() {  
  // Khởi động tất cả requests cùng lúc  
  const [user, posts] = await Promise.all([  
    fetch('/api/user'),  
    fetch('/api/posts')  
  ]);  
}
```

<div style="page-break-after: always;"></div>

Bài 4: Bảo mật cơ bản trong Giao tiếp Mạng

Bảo mật là yếu tố quan trọng hàng đầu trong lập trình mạng. Bài viết này sẽ giới thiệu các khái niệm và kỹ thuật bảo mật cơ bản khi phát triển ứng dụng web và mạng.

Tại sao cần Bảo mật Mạng?

Các mối đe dọa phổ biến

1. **Eavesdropping (Nghe lén):** Đọc dữ liệu truyền tải
2. **Man-in-the-Middle:** Chặn và sửa đổi dữ liệu
3. **Replay Attack:** Gửi lại request cũ
4. **Data Tampering:** Thay đổi dữ liệu

HTTP vs HTTPS

HTTP - Không an toàn

Dữ liệu truyền đi dưới dạng text thuần (Plain Text). Attacker có thể đọc được password, thông tin nhạy cảm nếu bắt được gói tin.

HTTPS - An toàn

Dữ liệu được mã hóa (Encrypted). Attacker bắt được gói tin cũng không

thể đọc được nội dung.

SSL/TLS là gì?

SSL (Secure Sockets Layer) và TLS (Transport Layer Security) là các giao thức mã hóa bảo vệ dữ liệu truyền tải.

Các phương pháp Mã hóa

1. Symmetric Encryption (Mã hóa đối xứng):

Cùng một key để mã hóa và giải mã. Tốc độ nhanh nhưng khó chia sẻ key an toàn.

Ví dụ: AES, DES.

2. Asymmetric Encryption (Mã hóa bất đối xứng):

Dùng cặp Public Key (để mã hóa) và Private Key (để giải mã).

Ví dụ: RSA, ECC.

3. Hashing (Băm):

Chuyển đổi một chiều, không thể giải mã ngược lại. Dùng để lưu password.

Ví dụ: SHA-256.

Các lỗ hổng bảo mật phổ biến

1. SQL Injection

// ✖ Nguy hiểm

```
const query = `SELECT * FROM users WHERE id = ${userId}`;
```

// ✔ An toàn - Parameterized Query

```
const query = 'SELECT * FROM users WHERE id = ?';
```

```
db.query(query, [userId]);
```

2. XSS (Cross-Site Scripting)

// ✖ Nguy hiểm

```
element.innerHTML = userInput;
```

```
// ✔ An toàn
```

```
element.textContent = userInput;
```

3. CSRF (Cross-Site Request Forgery)

Phòng chống bằng cách sử dụng **CSRF Token** xác thực mỗi request từ client.

```
<div style="page-break-after: always;"></div>
```

Bài 5: Bất đồng bộ và Xử lý đồng thời trong Lập trình

Trong lập trình hiện đại, việc xử lý nhiều tác vụ cùng lúc là điều cần thiết. Bài viết này sẽ giải thích khái niệm **bất đồng bộ (Asynchronous)** và **xử lý đồng thời (Concurrency)** trong JavaScript và Java.

Đồng bộ vs Bất đồng bộ

Xử lý đồng bộ (Synchronous)

Các tác vụ thực hiện **tuần tự**, tác vụ sau phải chờ tác vụ trước hoàn thành.

Xử lý bất đồng bộ (Asynchronous)

Các tác vụ có thể **chạy đồng thời**, không cần chờ nhau.

Bất đồng bộ trong JavaScript

JavaScript là **single-threaded** nhưng sử dụng **Event Loop** để xử lý bất đồng bộ.

Các cơ chế:

1. **Callback**: Cách cũ, dễ gây Callback Hell.
2. **Promise**: Đối tượng đại diện cho giá trị chưa có ở hiện tại.
3. **Async/Await**: Cú pháp "ngọt" (syntactic sugar) cho Promise.

```
console.log("Bắt đầu");

setTimeout(() => {
  console.log("Sau 2 giây");
}, 2000);

console.log("Tiếp tục");
// Output: Bắt đầu -> Tiếp tục -> Sau 2 giây
```

Xử lý đồng thời trong Java

Java sử dụng **Multi-threading** để xử lý đồng thời.

Tạo Thread

```
// Cách 1: Extend Thread

class MyThread extends Thread {
  @Override
  public void run() {
    System.out.println("Thread đang chạy");
  }
}

// Cách 2: Implement Runnable

class MyRunnable implements Runnable {
  @Override
  public void run() {
    System.out.println("Runnable đang chạy");
  }
}
```

Thread Pool

Sử dụng `ExecutorService` để quản lý luồng hiệu quả, tránh tạo/hủy luồng liên tục.

```
ExecutorService executor = Executors.newFixedThreadPool(4);  
executor.submit(() -> {  
    System.out.println("Task chạy trong thread pool");  
});
```

So sánh JavaScript vs Java

Khía cạnh	JavaScript	Java
Model	Single-threaded + Event Loop	Multi-threaded
Async	Promise, async/await	Future, CompletableFuture
Parallel	Worker threads (hạn chế)	Thread, ThreadPool

<div style="page-break-after: always;"></div>

Bài 6: DOM và Event trong Lập trình Web

DOM (Document Object Model) là giao diện lập trình cho phép JavaScript tương tác với cấu trúc HTML của trang web. Hiểu về DOM và Event là nền tảng quan trọng cho mọi lập trình viên web.

DOM là gì?

DOM biểu diễn trang web dưới dạng **cây các node (nodes tree)**. Mỗi phần tử HTML là một node trong cây.

```
document
├── html
│   ├── head
│   │   └── title
│   └── body
│       ├── h1
│       └── div
```

Truy cập phần tử DOM

// Chọn theo ID

```
const header = document.getElementById('header');
```

// Chọn theo CSS selector

```
const button = document.querySelector('.btn-primary');
```

```
const items = document.querySelectorAll('.item');
```

Thao tác với nội dung

```
const div = document.querySelector('#myDiv');
```

// Thay đổi nội dung text

```
div.textContent = 'Xin chào!';
```

```
// Thay đổi HTML
```

```
div.innerHTML = '<b>Bold</b>';
```

```
// Thay đổi CSS
```

```
div.style.color = 'red';
```

```
div.classList.add('active');
```

Events - Sự kiện

Event là các hành động xảy ra trên trang web (click, gõ phím, scroll, ...).

Cách gán Event

```
const button = document.getElementById('myButton');
```

```
// Cách chuẩn: addEventListener
```

```
button.addEventListener('click', function(event) {  
  console.log('Button clicked!');  
});
```

Các loại Event phổ biến

- **Mouse:** click, dblclick, mouseover
- **Keyboard:** keydown, keyup
- **Form:** submit, change, input
- **Window:** load, resize, scroll

Ví dụ Form Validation đơn giản

```
form.addEventListener('submit', (e) => {  
  e.preventDefault(); // Ngăn submit mặc định
```



```

if (username.value.length < 3) {
  alert('Tên quá ngắn!');
} else {
  // Gửi dữ liệu
}
});

```

```
<div style="page-break-after: always;"></div>
```

Bài 7: Fetch API và Giao tiếp HTTP trong JavaScript

Fetch API là giao diện hiện đại trong JavaScript để thực hiện các HTTP request. Nó thay thế XMLHttpRequest (XHR) cũ với cú pháp đơn giản hơn và dựa trên Promise.

Cú pháp cơ bản

```

fetch(url, options)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));

```

HTTP GET Request

GET dùng để **lấy dữ liệu** từ server:

```

async function layDanhSachSanPham() {
  try {
    const response = await
    fetch(['https://api.example.com/products'](https://api.example.com/products));

```

```

if (!response.ok) {
  throw new Error(`Lỗi HTTP: ${response.status}`);
}

const products = await response.json();
console.log('Sản phẩm:', products);
return products;

} catch (error) {
  console.error('Lỗi:', error);
}
}

```

HTTP POST Request

POST dùng để **gửi dữ liệu** tới server (ví dụ: tạo mới user):

```

async function taoNguoiDung(userData) {
  const response = await
  fetch(['https://api.example.com/users'](https://api.example.com/users)', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(userData)
  }));

  return await response.json();
}

```

Xử lý các loại Response

- `response.json()`: Parse JSON response.
- `response.text()`: Lấy text thuần.
- `response.blob()`: Lấy file (ảnh, pdf...).
- `response.formData()`: Lấy dữ liệu form.

HTTP Headers

```
const headers = {  
  'Content-Type': 'application/json',  
  'Authorization': 'Bearer my-token-123'  
};
```

<div style="page-break-after: always;"></div>

Bài 8: TCP và UDP: Hai giao thức vận chuyển cơ bản

Trong lập trình mạng, **TCP** và **UDP** là hai giao thức vận chuyển (Transport Layer) quan trọng nhất. Hiểu rõ đặc điểm và use case của từng giao thức giúp bạn thiết kế ứng dụng mạng hiệu quả.

TCP (Transmission Control Protocol)

Đặc điểm chính

1. **Hướng kết nối (Connection-oriented)**: Phải thiết lập kết nối (3-way handshake) trước khi truyền.
2. **Đáng tin cậy (Reliable)**: Đảm bảo dữ liệu đến đúng thứ tự, không mất gói.
3. **Kiểm soát luồng & tắc nghẽn**: Tự điều chỉnh tốc độ truyền.

Quy trình bắt tay 3 bước (Three-way Handshake)

1. **SYN**: Client gửi yêu cầu.
2. **SYN + ACK**: Server chấp nhận.

3. **ACK**: Client xác nhận -> Kết nối thiết lập.

UDP (User Datagram Protocol)

Đặc điểm chính

1. **Không hướng kết nối (Connectionless)**: Gửi là gửi, không cần bắt tay.
2. **Không đáng tin cậy (Unreliable)**: Có thể mất gói, gói đến sai thứ tự.
3. **Nhanh và nhẹ**: Header nhỏ (8 bytes) so với TCP (20+ bytes).

So sánh TCP vs UDP

Tiêu chí	TCP	UDP
Kết nối	Hướng kết nối	Không kết nối
Độ tin cậy	Cao (đảm bảo)	Thấp (không đảm bảo)
Thứ tự	Đúng thứ tự	Không đảm bảo
Tốc độ	Chậm hơn	Nhanh hơn
Use case	Web, Email, File transfer	Streaming, Gaming, VoIP

Khi nào dùng cái nào?

- **Dùng TCP** khi cần sự chính xác tuyệt đối (tải file, chat text, web).
- **Dùng UDP** khi cần tốc độ, chấp nhận mất mát nhỏ (video call, game online).

<div style="page-break-after: always;"></div>

Bài 9: WebSocket trong JavaScript: Giao tiếp Realtime

WebSocket là giao thức cho phép giao tiếp hai chiều (full-duplex) giữa client và server thông qua một kết nối TCP duy nhất. Đây là công nghệ quan trọng cho các ứng dụng realtime như chat, game online.

Tại sao cần WebSocket?

HTTP truyền thống hoạt động theo cơ chế Request-Response. Để cập nhật dữ liệu mới (ví dụ tin nhắn chat), client phải liên tục gửi request hỏi server (Polling), gây lãng phí tài nguyên và độ trễ cao.

WebSocket giải quyết bằng cách giữ một kết nối mở liên tục. Server có thể chủ động đẩy (push) dữ liệu xuống client ngay lập tức.

Sử dụng WebSocket trong JavaScript

Tạo kết nối

```
const socket = new WebSocket('ws://localhost:8080');
```

Các Event quan trọng

// 1. Khi kết nối thành công

```
socket.onopen = function(event) {  
  console.log('Đã kết nối!');  
  socket.send('Hello Server');  
};
```

// 2. Khi nhận được tin nhắn từ Server

```
socket.onmessage = function(event) {  
  console.log('Tin nhắn mới:', event.data);  
  const data = JSON.parse(event.data); // Parse nếu là JSON  
};
```

// 3. Khi có lỗi

```
socket.onerror = function(error) {  
  console.error('Lỗi WebSocket:', error);  
};
```

```
// 4. Khi kết nối đóng
socket.onclose = function(event) {
  console.log('Kết nối đã đóng');
};
```

Ứng dụng thực tế: Chat App

```
    // Gửi tin nhắn
function sendMessage(text) {
  const msg = {
    type: 'chat',
    content: text,
    timestamp: Date.now()
  };
  socket.send(JSON.stringify(msg));
}
```